1) What is Spring Security?

Interview Question

What is Spring Security and why is it used in Spring Boot applications?

Detailed Answer

Spring Security is a framework used to secure Spring applications by providing:

- **Authentication** → verifies who the user is
- **Authorization** → checks what the user is allowed to access
- **Protection against common attacks** → CSRF, session fixation, clickjacking, etc.

In Spring Boot applications, Spring Security is commonly used to:

- Protect REST APIs
- Restrict access based on roles/authorities
- Secure login flows
- Implement Basic Auth, Form Login, JWT, OAuth2, etc.

Core purpose:

When an HTTP request comes in, Spring Security intercepts it **before it reaches the controller** using a **filter chain**.

Then it checks:

1. Is the user authenticated?
2. Does the user have permission?

In modern projects:

For REST APIs, Spring Security is usually used with:

- **JWT-based stateless authentication**

Simple Explanation

Spring Security means:

"The security layer of a Spring application that handles login, access control, and request protection."

Example

Add dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Once added:

- All endpoints become secured by default.

Interviewer Cross-Question

Q: Does Spring Security work only for web applications?
A: No. It is commonly used in web apps and REST APIs, but it can also secure method invocations and application layers.

2) What is the difference between Authentication and Authorization?

Interview Question

What is the difference between authentication and authorization in Spring Security?

Detailed Answer

This is one of the most common interview questions.

Authentication

Authentication means **verifying the identity of the user**.

It answers:

- “Who are you?”

Examples:

- Username + password
- OTP

- JWT token validation
- OAuth login

If credentials are valid, the user becomes **authenticated**.

Authorization

Authorization happens **after authentication**.

It means checking whether the authenticated user has permission to access a resource.

It answers:

- "What are you allowed to do?"

Examples:

- USER can access `/profile`
- ADMIN can access `/admin`
- Only MANAGER can access `/reports`

Flow:

1. User sends credentials/token
2. Spring Security authenticates the user
3. Then it checks roles/authorities for access

Simple Explanation

- **Authentication** = Who are you?
- **Authorization** = What can you access?

Example

```
POST /login   -> Authentication
GET /admin    -> Authorization
```

Interviewer Cross-Question

Q: Can authorization happen without authentication?
A: Usually no, because the system must know who the user is before deciding permissions (except public endpoints).

3) What happens by default when we add Spring Security dependency?

Interview Question

What is the default behavior when `spring-boot-starter-security` is added to a Spring Boot application?

Detailed Answer

When we add the Spring Security starter dependency, Spring Boot auto-configures security.

Default behavior:

- **All endpoints are secured**
- A **default login page** is generated for web apps
- A default user is created:
    - username = `user`
    - password = generated and printed in console
- Unauthenticated requests are denied or redirected to login

For REST APIs:

If you access protected endpoints without authentication:

- You may get **401 Unauthorized**

Why is this useful?

It ensures the application is not accidentally left open.

Simple Explanation

After adding Spring Security:

“Everything becomes protected automatically unless you explicitly allow access.”

Example

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Then:

- `/hello`
- `/users`
- `/admin`

All require authentication by default.

Interviewer Cross-Question

Q: Where do we see the generated default password?
A: In the application startup console logs.

4) What is `SecurityFilterChain` and why is it important?

Interview Question

What is `SecurityFilterChain` in Spring Security, and why is it used instead of `WebSecurityConfigurerAdapter`?

Detailed Answer

`SecurityFilterChain` is the modern way to configure Spring Security in Spring Boot.

It defines:

- Which requests are secured
- Which endpoints are public
- Which authentication mechanisms to use

- CSRF rules
- Session rules
- Custom filters like JWT filters

Why important?

Spring Security works through a chain of filters that intercept requests before controllers.

Modern approach:

Older style:

```
extends WebSecurityConfigurerAdapter
```

❌ Deprecated

Current recommended approach:

Define a `@Bean` of type `SecurityFilterChain`

This is cleaner, more explicit, and aligns with modern Spring Boot configuration style.

Simple Explanation

`SecurityFilterChain` means:

“The main place where we define security rules for incoming HTTP requests.”

Example

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults());
```

```
    return http.build();
}
```

Interviewer Cross-Question

Q: Why was `WebSecurityConfigurerAdapter` deprecated?
A: Spring moved to a more component-based, bean-driven configuration model using `SecurityFilterChain`.

5) What is `UserDetailsService` and how is it used with JPA?

Interview Question

What is `UserDetailsService` in Spring Security, and how do you use it with Spring Data JPA?

Detailed Answer

`UserDetailsService` is a Spring Security interface used to load user details during authentication.

It contains one key method:

```
loadUserByUsername(String username)
```

This method is called when Spring Security needs to authenticate a user.

With JPA:

We typically:

1. Create a `UserEntity`
2. Create `UserRepository`
3. Implement `UserDetailsService`
4. Fetch user from DB
5. Convert it into Spring Security `UserDetails`

Why important?

This is the bridge between:

- Spring Security
- Your database (JPA)

This is exactly where your JPA knowledge becomes useful.

Simple Explanation

`UserDetailsService` means:

"A service that loads user information from DB (or memory) for login validation."

Example

```
@Service
public class CustomUserDetailsService implements
UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
throws UsernameNotFoundException {
        AppUser user =
userRepository.findByUsername(username)
                .orElseThrow(() -> new
UsernameNotFoundException("User not found"));

        return
org.springframework.security.core.userdetails.User
                .withUsername(user.getUsername())
                .password(user.getPassword())
                .roles(user.getRole())
                .build();
    }
}
```

Interviewer Cross-Question

Q: What should happen if user is not found?
A: Throw `UsernameNotFoundException`.

6) Why do we use `PasswordEncoder`?

Interview Question

Why is `PasswordEncoder` required in Spring Security, and which implementation is commonly used?

Detailed Answer

`PasswordEncoder` is used to securely hash passwords instead of storing them in plain text.

Why important?

Storing plain passwords is a major security risk.

Instead, passwords should be:

- Hashed
- One-way encoded
- Difficult to reverse

Common implementation:

- `BCryptPasswordEncoder`

Why BCrypt?

- Salted hashing
- Stronger than simple hashing
- Widely accepted standard
- Resistant to rainbow table attacks better than plain MD5/SHA usage

Flow:

1. User registers → password is encoded and stored
2. User logs in → raw password is encoded/compared against stored hash

Simple Explanation

`PasswordEncoder` means:

"Never store raw passwords. Always hash them securely."

Example

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Encoding password:

```
String encoded = passwordEncoder.encode("admin123");
```

Interviewer Cross-Question

Q: Can Spring Security compare raw password with encoded password automatically?
A: Yes, during authentication it uses the configured `PasswordEncoder`.

7) What is the difference between `hasRole()` and `hasAuthority()`?

Interview Question

What is the difference between `hasRole()` and `hasAuthority()` in Spring Security?

Detailed Answer

Both are used for authorization, but they differ slightly.

`hasRole("ADMIN")`

- Checks for role
- Spring automatically expects it internally as:

`ROLE_ADMIN`

So:

```
.hasRole("ADMIN")
```

means:

`ROLE_ADMIN`

`hasAuthority("READ_REPORTS")`

- Checks exact authority string
- No automatic prefix

Example:

```
.hasAuthority("READ_REPORTS")
```

Best interview answer:

- **Role is a higher-level grouping**
- **Authority is a more granular permission**
- In Spring Security, role is basically a special kind of authority with `ROLE_` prefix

Simple Explanation

- `hasRole("ADMIN")` → checks `ROLE_ADMIN`
- `hasAuthority("X")` → checks exact permission string

Example

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
```

```
    .requestMatchers("/reports/
**").hasAuthority("READ_REPORTS")
)
```

Interviewer Cross-Question

Q: If DB stores role as `ADMIN`, can `hasRole("ADMIN")` work?
A: Yes, if mapped properly because Spring internally prefixes role checks with `ROLE_`. But you should be consistent in how authorities are created.

8) What is JWT and why is it preferred for REST APIs?

Interview Question

What is JWT authentication, and why is it commonly used in Spring Boot REST APIs?

Detailed Answer

JWT stands for **JSON Web Token**.

It is a compact, signed token used to authenticate users in **stateless REST APIs**.

Why JWT is preferred for REST APIs?

Traditional session-based auth stores session state on the server.

But in REST/microservices:

- We often want **stateless authentication**
- The server should not store session for every client

JWT flow:

1. User sends username/password to login endpoint
2. Server validates credentials
3. Server generates JWT token
4. Client stores token
5. Client sends token in every request:

```
Authorization: Bearer <token>
```

6. Server validates token and allows access

Benefits:

- Stateless
- Scales better for distributed systems
- Good for React/Angular/mobile clients
- No server session storage required

JWT usually contains:

- Subject (username)
- Roles/claims
- Expiration time
- Signature

Simple Explanation

JWT means:

"After login, server gives a signed token. Client sends that token on every request instead of using server session."

Example Header

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Interviewer Cross-Question

Q: Is JWT encrypted by default?
A: Not necessarily. JWT is usually **signed**, not encrypted. Payload can often be decoded, so sensitive data should not be stored directly in it.

9) How does JWT authentication work in Spring Security?

Interview Question

How do you implement JWT authentication in Spring Security at a high level?

Detailed Answer

In Spring Security, JWT authentication usually works using a **custom filter**.

High-level flow:

Step 1: Login endpoint

- Client sends username + password
- Spring authenticates using `AuthenticationManager` / `UserDetailsService`
- If valid, server generates JWT token

Step 2: Client stores token

- Usually frontend stores it securely (implementation varies)

Step 3: Protected request

- Client sends:

```
Authorization: Bearer <token>
```

Step 4: JWT filter runs before controller

A custom filter (often extends `OncePerRequestFilter`) does:

1. Read Authorization header
2. Extract token
3. Validate token
4. Extract username/claims
5. Load user via `UserDetailsService`
6. Create `Authentication` object

7. Put it in `SecurityContextHolder`

Step 5: Request continues

Spring Security sees the user as authenticated and then checks authorization.

Simple Explanation

JWT in Spring Security means:

"A custom filter checks the token on every request and tells Spring Security who the user is."

Example (filter registration)

```
http.addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);
```

This means:

- JWT filter runs before the normal username/password filter

Interviewer Cross-Question

Q: Why do we add JWT filter before `UsernamePasswordAuthenticationFilter`?
A: Because we want token-based authentication to happen early before standard username/password processing.

10) Why do we disable CSRF and use `SessionCreationPolicy.STATELESS` in JWT APIs?

Interview Question

Why do we commonly disable CSRF and set session policy to stateless in JWT-based Spring Security?

Detailed Answer

This is a very common interview question for REST API security.

Why disable CSRF?

CSRF protection mainly matters when authentication relies on:

- Browser cookies
- Server-side sessions

In JWT-based REST APIs:

- We usually do **not** rely on session cookies
- The client explicitly sends token in header:

```
Authorization: Bearer <token>
```

So CSRF risk is different and traditional CSRF protection is usually unnecessary.

Therefore:

```
http.csrf(csrf -> csrf.disable());
```

Why set stateless session policy?

In JWT auth:

- Server should not create/store session
- Every request carries authentication info in the token

So we use:

```
.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS
))
```

This ensures:

- No HTTP session is created
- Authentication is fully token-driven
- Better for scalable REST APIs

Simple Explanation

For JWT APIs:

- Disable CSRF because we are not relying on session-cookie auth
- Set stateless because token is sent on every request

Example

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS
))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .anyRequest().authenticated()
        );

    return http.build();
}
```

Interviewer Cross-Question

Q: Is disabling CSRF always correct for every application?
A: No. It is commonly appropriate for **stateless JWT REST APIs**, but not necessarily for traditional session-based web apps.